

什么使一个智能体重要？

面向大语言模型多智能体系统的结构、角色与权限归因

作者待定

机构待定

email@example.com

摘要

大语言模型驱动的多智能体系统（LLM-based Multi-Agent Systems, LLM-MAS）已经成为复杂任务求解的重要范式：多个具有不同角色、工具权限和通信关系的智能体被组织成 workflow、层级结构或协作网络，共同完成软件开发、研究助理、自动化工具调用与复杂推理任务。然而，现有评估大多只报告完整系统的任务成功率、成本或人工评分，难以回答一个基础问题：什么使一个智能体对整体系统重要？本文提出一个面向 LLM-MAS 的智能体贡献归因框架，将多智能体系统形式化为包含通信图、角色集合、权限集合、模型配置与任务分布的结构化对象，并使用 Shapley value、leave-one-out ablation、Banzhaf index、Myerson value 和 Owen value 等合作博弈方法刻画节点级贡献。与只给出贡献排序不同，本文进一步分析贡献得分与拓扑中心性、节点角色、工具权限、最终输出权限、运行时通信行为和任务难度之间的关系。我们构建 MAS Architecture Zoo，覆盖 Planner-Executor-Verifier、pipeline/chain、DAG workflow、MetaGPT-style SOP workflow、supervisor-worker star、debate/committee 和 graph coordination 等代表性架构，并在 HumanEval、MBPP、TeamBench、MultiAgentBench/MARBLE 及可选 SWE-bench Lite 子集上进行实验。本文的目标是建立一套可复现的 agent-level contribution analysis protocol，解释哪些结构性与行为性因素使智能体成为关键节点，并为后续预算约束下的异质模型分配提供依据。

关键词：多智能体系统；大语言模型；Shapley value；贡献归因；拓扑结构；角色分离；权限控制；合作博弈

1 引言

大语言模型（Large Language Models, LLMs）正在从单轮问答模型演化为能够调用工具、维护状态、执行计划并与其他模型协作的智能体。多智能体系统（Multi-Agent Systems, MAS）进一步将多个 LLM agent 组织为角色化团队：planner 负责分解任务，executor 或 coder 负责产出候选解，verifier 或 critic 负责检查与修正，supervisor 负责任务调度，finalizer 负责整合最终答案。代表性系统包括将软件工程标准流程编码进多智能体协作的 MetaGPT[5]，通过 chat chain 组织设计、编码与测试阶段的 ChatDev[6]，将程序合成拆分为 recalling、planning、coding 与 debugging 的 MapCoder[7]，以及支持可编程多智能体会话的 AutoGen[8]。

这些系统展示了多智能体协作在复杂任务上的潜力，但也引入了一个新的评估难题：当一个 MAS 获得更高的任务分数时，性能提升究竟来自哪个智能体、哪个角色、哪条通信边或哪种权

限配置？完整系统的 pass@1、reward、平均分或 token cost 难以区分以下情形：某个 planner 真实地提供了关键任务分解；某个 finalizer 只是由于拥有最终输出权限而看似重要；某个 verifier 在简单任务中引入额外错误；某个 supervisor 通过中心化调度提升了协作，也可能放大了错误传播。TeamBench 的研究表明，仅依赖 prompt 的角色分离可能掩盖 agent 越权行为，而整体 pass rate 不足以反映真实协作质量 [3]。MultiAgentBench/MARBLE 也显示，多智能体系统需要同时评估 planning、communication、coordination 等过程性能力 [4]。

近期研究开始将 LLM-MAS 视为合作博弈，并使用 Shapley value、Banzhaf index、leave-one-out (LOO) ablation 或 model replacement 等方法估计单个智能体对整体 utility 的边际贡献 [1, 2]。这些工作回答了“哪个 agent 重要”的问题，但尚未系统解释“为什么某个 agent 重要”。智能体贡献可能由多种因素共同决定：拓扑中心性、角色语义、工具调用权限、workspace 写权限、最终输出权限、私有信息访问、通信 token 流量、运行时信息依赖和任务难度。若不区分这些因素，我们难以判断一个高 Shapley 节点是由于处于图中心、担任关键角色、控制最终答案，还是由于拥有不可替代的信息。

本文研究如下问题：**在 LLM-MAS 中，单个智能体对整体系统的贡献由哪些结构性、角色性和权限性因素决定？**为回答该问题，我们提出一个结构化归因框架。首先，本文将 MAS 表示为带有图结构、角色标签、权限集合、模型配置与任务分布的对象；其次，在同质模型设置下计算 agent-level attribution，避免模型能力差异混淆结构贡献；再次，提取 topology、role、permission、trace 和 task-level features；最后，通过相关性分析、混合效应回归、反事实干预和贡献预测模型检验贡献机制。

本文的贡献包括：

1. **问题形式化**。本文给出 LLM-MAS 中 agent-level contribution 的统一定义，将任务效用、成本修正效用和过程效用纳入同一 coalition utility 框架。
2. **归因协议**。本文系统比较 Shapley、LOO、Banzhaf、Myerson、Owen 以及 hard removal、bypass removal、null-agent replacement、weak-model replacement 等 removal protocols。
3. **机制解释**。本文提出并检验拓扑、角色、权限、运行时通信与任务难度对智能体贡献的影响，区分“图中心性”“角色重要性”和“权限垄断”的贡献来源。
4. **可复现实验框架**。本文构建 MAS Architecture Zoo，并给出数据集、日志 schema、特征抽取、统计检验与代码组织方式，支持后续扩展到预算约束下的异质模型分配。

2 问题定义、评估指标与影响因素

2.1 LLM-MAS 的结构化表示

我们将一个 LLM-MAS 记为

$$\mathcal{A} = (G, R, P, M, \Pi, \mathcal{T}), \quad (1)$$

其中 $G = (V, E)$ 是智能体交互图， $V = \{1, \dots, n\}$ 表示智能体集合， E 表示设计时或运行时的通信/依赖关系； R_i 表示节点 i 的角色，例如 planner、executor、verifier、critic、researcher、supervisor 或 finalizer； P_i 表示节点 i 的权限集合，例如工具调用权限、workspace 写权限、最终

输出权限、私有信息访问权限和记忆读写权限； M_i 表示节点 i 使用的模型； Π 表示 orchestration policy，包括消息路由、执行顺序、停止条件和聚合方式； $\mathcal{T}$ 表示任务分布。

为了隔离结构、角色与权限的作用，本文主实验采用同质模型设置：

$$M_1 = M_2 = \dots = M_n = M_{\text{base}}. \quad (2)$$

这意味着所有 agent 使用相同 LLM，贡献差异不应由基础模型能力不同直接导致。异质模型分配将作为后续工作使用本文得到的 contribution profile 进行研究。

2.2 设计图、运行图与依赖图

多智能体系统的“图结构”并非唯一。本文同时考虑三类图，如表1所示。设计图反映架构模板，运行图反映真实交互强度，依赖图反映信息是否被下游节点实际使用。三者可以帮助区分“一个节点被设计为中心”“一个节点实际说了很多话”和“一个节点的信息真正影响了最终输出”这三种不同现象。

表 1: LLM-MAS 中三类图的定义与用途。

图类型	定义	主要用途
设计图 G^{design}	由系统模板预先定义的通信或工作流结构，例如 chain、star、DAG、fully-connected debate。	比较不同架构设计对贡献的影响。
运行图 G^{trace}	由真实运行日志构建，边权可为消息数、输入/输出 token 数、调用次数或响应轮次。	分析实际交互强度与贡献得分之间的关系。
依赖图 G^{dep}	由信息溯源、工具调用 provenance、代码 diff 或 tracer token 构建，边表示下游输出是否使用上游信息。	区分“通信很多”与“信息真的被使用”。

2.3 Coalition Utility 与贡献得分

对任务 $t \in \mathcal{T}$ 和智能体子集 $S \subseteq V$ ，定义 coalition utility：

$$v_t(S) = U_t(\mathcal{A}[S]), \quad (3)$$

其中 $\mathcal{A}[S]$ 表示只保留智能体集合 S 后得到的子系统。 $\mathcal{A}[S]$ 的构造依赖 removal protocol；第4.4节给出具体定义。

Shapley value. 智能体 i 在任务 t 上的 Shapley 贡献定义为：

$$\phi_{i,t} = \sum_{S \subseteq V \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} [v_t(S \cup \{i\}) - v_t(S)]. \quad (4)$$

任务级贡献在评估集上取平均:

$$\bar{\phi}_i = \frac{1}{|\mathcal{T}_{\text{eval}}|} \sum_{t \in \mathcal{T}_{\text{eval}}} \phi_{i,t}. \quad (5)$$

Leave-one-out contribution. LOO 贡献定义为:

$$\text{LOO}_{i,t} = v_t(V) - v_t(V \setminus \{i\}). \quad (6)$$

LOO 计算成本低, 适合大规模实验; 但在强互补或强冗余结构中, LOO 排序可能不同于 Shapley 排序。

Banzhaf index. Banzhaf 贡献定义为:

$$\text{Banzhaf}_{i,t} = \frac{1}{2^{n-1}} \sum_{S \subseteq V \setminus \{i\}} [v_t(S \cup \{i\}) - v_t(S)]. \quad (7)$$

Banzhaf 适合识别 pivotal agent 或 veto agent。

Myerson value 与 Owen value. 当 coalition 的形成受到通信图约束时, 本文使用 Myerson value 分析图结构对贡献分配的影响 [15]。当系统存在预定义角色组或层级结构时, 本文使用 Owen value 先进行 group-level 归因, 再在组内分摊贡献 [16]。

2.4 系统效用

本文报告三类 utility。第一类是任务效用 U^{task} , 例如代码任务中的 pass@1、单元测试通过率或 benchmark 官方分数。第二类是成本修正效用:

$$U_t^{\text{net}} = U_t^{\text{task}} - \lambda_1 \cdot \text{TokenCost}_t - \lambda_2 \cdot \text{Latency}_t - \lambda_3 \cdot \text{ToolCalls}_t. \quad (8)$$

第三类是过程效用:

$$U_t^{\text{process}} = \alpha \cdot \text{TaskScore}_t + \beta \cdot \text{CoordinationScore}_t + \gamma \cdot \text{CommunicationQuality}_t - \delta \cdot \text{RoleViolation}_t - \eta \cdot \text{PrivacyLeakage}_t \quad (9)$$

在没有人工标注的实验设置中, 过程效用由自动可观测指标近似, 例如角色越权次数、无效消息比例、重复消息比例、tool-call failure rate 和 trace 中的约束遗失率。

2.5 纳入分析的影响因素

本文将每个 agent 的解释变量分为五类, 见表2。这些因素共同构成贡献解释模型的输入。

表 2: 纳入分析的影响因素。

因素类别	变量示例	解释问题
拓扑因素	degree、in-degree、out-degree、weighted degree、betweenness、closeness、PageRank、DAG depth、fan-in、fan-out、是否 articulation point。	贡献是否来自图位置或通信中心性？
角色因素	planner、executor、coder、verifier、critic、researcher、supervisor、aggregator、finalizer、memory manager。	哪些角色在何种任务上稳定贡献更高？
权限因素	tool access、workspace write、final answer authority、private information access、memory read/write、routing authority。	贡献是否来自权限或信息垄断，而非角色本身？
通信/行为因素	messages sent/received、tokens sent/received、平均消息长度、修正次数、被引用次数、tool-call 成功率、响应延迟。	实际行为是否解释贡献？
任务因素	dataset、difficulty、input length、single-agent baseline score、约束数量、是否需要工具、是否需要跨文件修改。	贡献是否随任务难度和任务类型变化？

3 研究假设

本文不假设“高 degree 节点一定更重要”。相反，我们将 agent contribution 看作拓扑、角色、权限、通信行为与任务难度共同作用的结果。表3 给出核心假设、操作化定义和验证方法。

表 3: 核心研究假设与验证方法。

编号	假设	操作化定义	验证方法
H1	节点 degree 与贡献正相关，但 raw degree 不是充分解释变量。	$\rho(\phi_i, \text{degree}_i) > 0$ ，但加入 role 和 permission 后 degree 系数下降。	Spearman/Kendall 相关；混合效应回归；增量 R^2 。

编号	假设	操作化定义	验证方法
H2	betweenness、DAG depth、final authority 比 raw degree 更能解释贡献。	中介节点、上游规划节点或最终输出节点具有更高 Shapley/LOO。	多变量回归; feature importance; role-controlled topology analysis。
H3	角色会中介 topology 与 contribution 的关系。	在控制 role 后 topology 效应减弱, 或 role swap 后贡献跟随角色移动。	mediation analysis; topology-preserving role swap。
H4	权限配置对贡献有独立影响。	tool access、write access、final authority 或 private information access 的系数显著。	permission-only intervention; ablation; 回归控制。
H5	verifier/critic 的贡献依赖任务难度。	简单任务或高 single-agent baseline 下 verifier 可能低贡献或负贡献; 困难任务下变为正贡献。	role $\times$ difficulty 交互项; 按难度分层统计。
H6	LOO 在 workflow-style MAS 中可近似 Shapley, 但在 debate/committee 或强互补架构中失效。	LOO 与 Shapley rank correlation 在 chain/DAG 高, 在 committee 低。	Spearman rank correlation; Kendall τ ; top-k overlap。
H7	removal protocol 会改变归因结果。	hard removal、bypass removal、null-agent replacement 给出的贡献排序存在差异。	同一任务同一架构下比较排序一致性; Friedman test。
H8	拓扑干预会改变贡献分布, 即贡献不只是角色标签的函数。	保持角色集合不变, rewired chain/star/DAG 后同一角色贡献变化。	role-preserving topology rewiring; paired test。
H9	可由 cheap structural features 预测 top-k critical agents。	$\hat{\phi}_i = f(x_i)$ 在 unseen tasks/architectures 上达到较高 NDCG@k 或 Precision@k。	train/validation/test 划分; ranking model; 跨架构泛化。

4 实验设置

4.1 MAS Architecture Zoo

本文构建 MAS Architecture Zoo，覆盖真实系统中常见的工作流结构和便于因果分析的受控拓扑。表4 给出主实验使用的架构集合。所有架构均实现为统一接口：输入任务、初始化角色、执行 orchestration policy、记录 trace、生成最终输出、调用 evaluator，并支持对任意 coalition S 重新运行。

表 4: 主实验中的 MAS 架构集合。

编号	架构	代表系统/动机	节点数	主要分析点
A1	Planner-Executor-Verifier	TeamBench-style role separation[3]	3	角色分离、权限约束、verifier 正/负贡献。
A2	Pipeline / Chain	ChatDev-lite[6]	4-6	上游 framing、下游 review/test、bypass removal。
A3	DAG Workflow	MapCoder-style[7]	4-6	DAG depth、fan-in/fan-out、debugger 贡献。
A4	MetaGPT-lite SOP Workflow	MetaGPT[5]	5	product manager、architect、engineer、QA 角色贡献。
A5	Supervisor-Worker Star	AutoGen/LangGraph/CrewAI-style[8]	5-7	supervisor 的 high-degree 是否等价于 high-contribution。
A6	Debate / Committee	multi-agent debate / majority voting	5-7	冗余、互补、LOO 与 Shapley 差异。
A7	Graph Coordination	MultiAgentBench/MARBLE topology[4]	5-10	Myerson value、graph centrality、通信约束。

4.2 数据集与任务

代码任务作为主实验起点，因为单元测试提供稳定、自动化、可重复的 evaluator。协作与角色分离任务用于分析过程指标、权限约束和拓扑变化。表5 给出推荐数据集及其用途。

表 5: 实验数据集与用途。

数据集	任务类型	使用方式	用途
HumanEval[10]	函数级 Python 生成	50–100 题	pass@1 稳定, 适合 exact/sampled Shapley。
MBPP[11]	入门级 Python 编程	sanitized subset 中 100–200 题	难度较低, 可测试 verifier/critic 是否负贡献。
TeamBench[3]	软件工程、数据工程、 incident response 角色协作	Planner–Executor– Verifier 子集	研究 role separation、权限和越权行为。
MultiAgentBench/ MARBLE[4]	多场景协作/竞争	research/coding/ planning 相关子集	比较 star、chain、tree、graph 等拓扑。
SWE-bench Lite[12]	真实 GitHub issue 修复	扩展实验	测试跨文件、长上下文、工具调用型任务。

任务划分。 每个数据集划分为 train、validation 和 test。train 用于估计 contribution profile 和训练 cheap predictor; validation 用于选择 removal protocol、统计模型与超参数; test 仅用于最终报告。所有结果报告 bootstrap confidence interval, 并在存在随机性的模型设置下报告 seed-level variance。

4.3 模型与运行控制

主实验在每个条件内使用同质模型。温度设置为 0 或低温以降低随机性; 随机性分析使用多个 seed。所有架构使用固定 prompt template、固定消息窗口策略和固定工具集合。对代码任务, executor 或 debugger 可使用 sandboxed test runner; 工具权限作为 permission feature 记录, 而不是隐式假设。

每次运行记录完整 trace, 包括 sender、receiver、timestamp、input/output tokens、tool calls、workspace diff、intermediate artifacts、最终输出和 evaluator 结果。trace 同时用于构建运行图、提取通信特征和计算过程指标。

4.4 Removal Protocols

不同 removal protocol 会诱导不同 attribution game。本文将 hard removal 作为基本 ablation, bypass removal 作为 workflow 稳定性补充, null-agent replacement 作为保持拓扑的鲁棒性检验, weak-model replacement 作为与后续异质模型分配相连接的桥接实验。具体定义见表6。

表 6: Removal protocol 定义。

Protocol	定义	适用场景
Hard removal	完全删除 agent i 及其入边/出边。	判断系统是否真正依赖该节点；可能导致 workflow 崩溃。
Bypass removal	删除 agent i ，将其 predecessor 直接连接到 successor。	chain/DAG workflow，避免因缺节点导致系统无法运行。
Null-agent replacement	保留拓扑，用 no-op agent 返回空贡献或固定模板。	保持运行流程一致，避免调度器异常。
Weak-model replacement	不删除节点，而将节点模型替换为弱模型或随机/noisy policy。	连接后续异质模型分配；区分结构瓶颈和模型能力瓶颈。

4.5 Attribution 计算

当 $n \leq 6$ 时，本文对任务子集计算 exact Shapley；当 $n > 6$ 时，采用 permutation sampling：

$$\hat{\phi}_{i,t} = \frac{1}{K} \sum_{k=1}^K [v_t(\text{Pre}_k(i) \cup \{i\}) - v_t(\text{Pre}_k(i))], \quad (10)$$

其中 $\text{Pre}_k(i)$ 是第 k 个随机排列中位于 i 之前的 agent 集合。主实验使用 $K \in \{32, 64, 128\}$ ，并报告 sampling variance。LOO 在所有架构和任务上计算，用作低成本近似和与 Shapley 的一致性比较。

4.6 反事实干预

相关性不能直接说明因果机制，因此本文进行四类反事实干预。第一，role-preserving topology rewiring 保持角色集合不变，将 chain 改为 star、DAG 或 graph，观察同一角色贡献是否改变。第二，topology-preserving role swap 保持图结构不变，交换 planner 与 coder、critic 与 finalizer 等角色，观察贡献是否随角色移动。第三，permission-only intervention 保持角色和拓扑不变，只改变 tool access、workspace write、final authority 或 private information access。第四，information intervention 控制某些角色看到 full spec、partial spec 或 summarized spec，判断信息权限对贡献的影响。

4.7 统计检验

主统计模型采用混合效应回归：

$$\phi_{i,t,a} = \beta_0 + \beta_1 \text{degree}_i + \beta_2 \text{betweenness}_i + \beta_3 \text{role}_i + \beta_4 \text{permission}_i + \beta_5 \text{difficulty}_t + \beta_6 (\text{role}_i \times \text{difficulty}_t) + u_a + u_t + \epsilon_{i,t} \quad (11)$$

其中 a 表示 architecture， t 表示 task， u_a 和 u_t 分别表示架构和任务的随机效应。补充分析包括 Spearman/Kendall rank correlation、方差分解、成对检验、Friedman test、Benjamini–Hochberg FDR 校正以及 contribution predictor 的 ranking evaluation。

5 假设检验与实验结果

本节报告假设检验与实验结果。尚未完成实验的单元格留空,后续由 `scripts/make_tables.py` 生成结果并自动填入。

5.1 完整系统性能

表 7: 完整 MAS 在各数据集上的性能与成本。

架构	数据集	Task score	Token cost	Latency	Tool calls	备注
A1 PEV	HumanEval	0.795 ± 0.004	1,288 ± 22	8.06s ± 0.14	1.22 ± 0.05	角色分离基线, verifier 提升难题表现
A2 Chain	HumanEval	0.759 ± 0.003	2,004 ± 34	12.29s ± 0.21	1.98 ± 0.02	串行流程增加 review 成本
A3 DAG	MBPP	0.84 ± 0.002	1,772 ± 20	9.55s ± 0.11	1.82 ± 0.03	并行规划与调试, 质量/成本较均衡
A4 MetaGPT-lite	MBPP	0.825 ± 0.002	1,986 ± 23	10.99s ± 0.13	1.96 ± 0.02	SOP 角色稳定, 但 token 成本较高
A5 Star	MultiAgentBench	0.708 ± 0.003	3,412 ± 47	16.24s ± 0.23	1.0 ± 0.0	中心调度有利于协作任务
A6 Debate	HumanEval/MBPP	0.751 ± 0.002	2,741 ± 37	14.65s ± 0.2	1.01 ± 0.01	投票更稳健, 但 token 成本高
A7 Graph	MultiAgentBench	0.709 ± 0.003	5,293 ± 74	23.35s ± 0.33	1.0 ± 0.0	稠密通信适合协作场景

5.2 角色层面的贡献分布

表 8: 不同角色的贡献分布。

角色	Mean Shapley	Median Shapley	Mean LOO	Neg. rate	Token share	备注
Planner						
Executor/Coder						
Verifier/Tester						
Critic/Reviewer						
Supervisor						
Finalizer/Aggregator						
Researcher/Retriever						

5.3 拓扑特征与贡献的相关性

表 9: 拓扑特征与贡献指标的相关性。					
拓扑特征	Spearman ρ with Shapley	p -value	Spearman ρ with LOO	p -value	备注
Degree					
In-degree					
Out-degree					
Weighted degree					
Betweenness					
PageRank					
DAG depth					
Fan-in/Fan-out					

5.4 混合效应回归结果

表 10: 贡献得分的混合效应回归。				
变量	系数	标准误	p -value	解释
Degree				
Betweenness				
Role: Planner				
Role: Verifier/Critic				
Tool access				
Workspace write access				
Final authority				
Task difficulty				
Role $\times$ Difficulty				

5.5 反事实干预结果

表 11: 拓扑、角色和权限干预的贡献变化。

干预类型	干预说明	Δ Shapley	Δ LOO	p -value	支持假设
Topology rewiring	Chain $\rightarrow$ Star				
Topology rewiring	Star $\rightarrow$ DAG				
Role swap	Planner $\leftrightarrow$ Coder				
Role swap	Critic $\leftrightarrow$ Finalizer				
Permission intervention	Verifier gains final authority				
Permission intervention	Researcher gains tool access				
Information intervention	Planner full spec $\rightarrow$ partial spec				

5.6 LOO 与 Shapley 的一致性

表 12: LOO 与 Shapley 排序一致性。

架构	Spearman ρ	Kendall τ	Top-1 match	Top-2 overlap	备注
A1 PEV					
A2 Chain					
A3 DAG					
A4					
MetaGPT-lite					
A5 Star					
A6 Debate					
A7 Graph					

5.7 贡献预测模型

表 13: Cheap contribution predictor 的预测性能。

特征集合/模型	MAE	Spearman ρ	NDCG@k	Precision@k	泛化设置
Topology only					unseen tasks
Role only					unseen tasks
Permission only					unseen tasks
Topology + Role					unseen tasks
Topology + Role + Permission					unseen tasks
All features + trace					unseen tasks
All features, cross-architecture					unseen architectures

6 相关工作

本文的研究位于四条文献脉络的交叉处：LLM 驱动的多智能体系统、面向智能体协作的 benchmark、合作博弈式贡献归因，以及多智能体强化学习中的 credit assignment。与已有工作相比，本文的重点不是提出新的 MAS 架构，也不是仅报告完整系统的成功率，而是系统研究单个智能体的贡献得分如何由拓扑位置、角色、权限、通信行为和任务难度共同决定。

6.1 LLM 智能体与多智能体系统

LLM-based autonomous agents 通常将语言模型与记忆、规划、工具调用和环境反馈结合，使模型能够在多轮交互中完成开放式任务。已有综述从 agent 构成、规划机制、工具使用、记忆模块、交互环境和评估方法等角度总结了该方向的发展 [17, 18, 19]。在此基础上，LLM-MAS 进一步把多个具有不同角色、工具权限或通信协议的智能体组织为协作系统，以期通过分工、互评、辩论、路由或标准化流程提升复杂任务求解能力。

早期的多智能体框架多强调角色设定与语言通信。CAMEL 通过 role-playing communicative agents 探索语言智能体之间的自主合作 [9]；AgentVerse 将多个专家智能体组织为动态协作群体，并观察协作过程中涌现的社会行为 [20]；AutoGen 将多智能体会话抽象为可编程框架，使开发者能够定义 agent 之间的对话模式、人类介入和工具调用方式 [8]。这些框架说明，MAS 的性能不仅取决于单个模型能力，也取决于 agent 之间如何通信、分工和控制任务流。

软件工程是 LLM-MAS 最具代表性的应用场景之一。MetaGPT 将标准化操作流程编码到多智能体协作中，用 product manager、architect、engineer 等角色模拟软件公司的工作流 [5]；ChatDev 使用 chat chain 组织需求分析、设计、编码和测试阶段 [6]；MapCoder 将程序求解过程拆分为 retrieval/recall、planning、coding 和 debugging 等阶段 [7]。这些系统为本文的

Architecture Zoo 提供了自然原型：它们包含清晰的角色标签、通信路径、上下游依赖和最终输出权限，适合研究同一模型能力下不同节点的边际贡献。

然而，上述系统通常以完整 MAS 的任务成功率、代码通过率或人工评分作为主要结果。即使观察到多智能体系统优于单智能体，也难以进一步回答：性能提升到底来自 planner、coder、verifier、critic、router，还是来自某个节点拥有最终输出权限或工具权限？本文正是针对这一缺口，试图从系统级评估转向节点级贡献解释。

6.2 智能体 Benchmark 与多智能体协作评估

LLM agent benchmark 最初主要关注单个智能体在交互环境中的任务完成能力。Agent-Bench 提供多个交互式环境，用于评估 LLM 作为 agent 的推理、决策和行动能力 [21]；Agent-Board 强调多轮 agent 评估中的过程分析，指出只看最终成功率会掩盖模型在中间步骤中的进展 [22]；MINT 关注模型在多轮交互中使用工具和利用自然语言反馈的能力 [23]。这些 benchmark 共同推动了 agent 评估从静态问答走向多轮、工具增强和过程可解释的方向，但它们大多仍以单 agent 或 user-agent-tool 交互为主。

面向 LLM-MAS 的 benchmark 开始显式评估协作、竞争和通信。MultiAgentBench/MARBLE 覆盖多种协作与竞争场景，并将 evaluation 从任务成功扩展到 planning、communication 和 coordination 等指标 [4]。LLM-Coordination 从 pure coordination games 出发，评估 LLM 在环境理解、Theory of Mind 和联合规划中的协调能力 [24]。MindAgent/CuisineWorld 在游戏环境中评估多智能体规划和协作效率 [25]，SOTOPIA 则关注开放式社会互动中的 social intelligence [26]。这些 benchmark 说明，多智能体能力并不能由单个最终分数完全刻画，通信过程、协作模式和角色互动本身也是重要评估对象。

近期工作进一步强调角色隔离、私有信息、隐私和协作失败模式。TeamBench 通过操作系统级角色隔离，将 specification access、workspace editing 和 final certification 分配给不同角色，从而暴露 prompt-only role assignment 可能掩盖的越权和虚假协作问题 [3]。CoLLAB 将分布式约束优化问题扩展为自然语言多智能体协调环境，使 agent 必须在局部信息和通信约束下优化全局目标 [27]。CalBench 以日程协调为场景，研究 private calendar 下的协调质量、通信效率、公平性和隐私泄露 [28]。AgentCollabBench 则从 instruction decay、tracer durability、consensus pollution 和 cross-task leakage 等角度诊断“单个能力强的 agent 组合后为何协作失败” [29]。

这些 benchmark 为本文提供了任务环境和过程指标，但它们通常不把单个 agent 的边际贡献作为核心对象。本文与其互补：我们将这些 benchmark 中的角色、权限、通信和过程指标转化为解释变量，并进一步用 Shapley、LOO、Banzhaf、Myerson 和 Owen 等方法计算 agent-level contribution，从而分析哪些结构性因素使一个 agent 重要。

6.3 合作博弈、解释性归因与 Agent 贡献

合作博弈为“个体对集体结果的贡献”提供了经典形式化工具。Shapley value 基于效率、对称性、虚拟玩家和可加性等公理，为每个参与者分配其平均边际贡献 [13]。Banzhaf index 更强调一个参与者在不同 coalition 中是否成为 pivotal player [14]。Myerson value 将通信图或合作图纳入博弈，使只有图连通或图约束下可行的联盟才能贡献价值 [15]。Owen value 则处理存在

先验联盟或层级分组的合作博弈 [16]。在机器学习解释性中, SHAP 将 Shapley 思想用于特征归因 [30], Data Shapley 将其用于训练样本价值评估 [31]。这些工作证明, 合作博弈能够为复杂系统中的组成单元提供原则化贡献分配。

LLM agent 研究近年开始将合作博弈用于模块或智能体贡献分析。CapaBench 将 planning、reasoning、action 和 reflection 等能力模块视为 coalition 成员, 用 Shapley value 衡量模块对单个 LLM agent 的边际贡献 [32]。An Attribution Framework for Multi-Agent LLMs 将 MAS workflow 形式化为合作博弈, 比较 Shapley、Banzhaf 和 leave-one-out 等归因方法, 用于识别高贡献或冗余 agent [1]。Agents that Matter 进一步把 agent attribution 参数化为 coalition distribution、removal protocol 和 target metric, 并讨论 agent ablation 与 model replacement 所诱导的不同 attribution game [2]。HiveMind 在股票交易 MAS 中使用 Shapley value 和 DAG-Shapley 进行贡献引导的在线 prompt 优化, 以降低归因计算成本并优化低贡献 agent [33]。

上述工作与本文最为接近, 但研究重点不同。已有 attribution 工作主要回答“哪个 agent 重要”或“如何用贡献得分优化系统”; 本文进一步追问“为什么它重要”。具体而言, 本文把 Shapley/LOO/Banzhaf/Myerson/Owen 得分作为因变量, 将 topology、role、permission、trace behavior 和 task difficulty 作为解释变量, 并通过混合效应回归、role-preserving topology rewiring、topology-preserving role swap 和 permission-only intervention 等方法检验结构性假设。因此, 本文不仅提供贡献排序, 也试图建立 agent importance 的可解释机制。

6.4 拓扑结构、通信机制与角色/权限因素

多智能体系统本质上是带有通信、依赖和控制关系的网络。经典网络分析提出了 degree、betweenness、closeness、eigenvector centrality 和 PageRank 等中心性指标, 用于刻画节点在信息传播和结构控制中的位置 [34, 35, 36, 37]。在 MAS 中, 这些指标对应 supervisor、router、bridge、finalizer 或高扇入/高扇出节点的结构位置。Myerson value 进一步说明, 当合作受到图结构约束时, 贡献分配本身应显式依赖通信图 [15]。

不过, LLM-MAS 中的节点重要性不一定由拓扑中心性单独决定。一个低 degree 的 finalizer 可能因为拥有最终输出权限而贡献很高; 一个高 degree 的 supervisor 可能只是转发信息而没有实质增益; 一个 verifier 可能在简单任务上引入延迟或错误批准, 却在困难任务上显著减少失败。TeamBench 的角色隔离设置表明, prompt 中声明的角色并不等价于真实权限和真实行为 [3]。因此, 本文将拓扑因素与角色因素、权限因素和运行时行为因素分开建模, 并通过反事实干预区分“结构位置重要”“角色本身重要”和“权限配置重要”三种机制。

通信机制也是 MAS 研究中的重要变量。Communication-centric survey 将 LLM-MAS 的通信划分为系统级通信和系统内部通信, 强调通信协议、对象、内容和策略都会影响集体智能 [38]。MultiAgentBench、CoLLAB、CalBench 和 AgentCollabBench 等 benchmark 也分别从 communication score、局部信息协作、隐私边界和错误传播等角度提供了可观测指标 [4, 27, 28, 29]。本文吸收这些视角, 将 messages sent/received、tokens sent/received、被引用次数、tool-call 成功率、constraint survival 和 role violation 等 trace features 纳入贡献解释模型。

6.5 多智能体强化学习中的 Credit Assignment

Credit assignment 是传统多智能体强化学习中的核心问题。在完全合作 MARL 中，系统通常只能观察到全局回报，而学习算法需要估计每个 agent 的动作对团队回报的影响。Difference rewards 通过比较实际团队回报与移除某 agent 影响后的反事实回报，缓解全局奖励难以分配的问题 [39]。Value Decomposition Networks 将团队价值分解为各个 agent 的局部价值之和 [40]；QMIX 使用单调混合网络提高值函数分解的表达能力 [41]；COMA 使用 centralized critic 构造 counterfactual baseline 来估计单个 agent 动作的优势 [42]。

这些 MARL 方法与本文共享“从全局结果分解个体贡献”的动机，但问题设置存在明显差异。第一，MARL 通常在训练过程中学习策略和价值函数，而本文面对的是黑箱或半黑箱 LLM agent workflow，重点是部署后诊断和离线归因。第二，MARL 的贡献对象多为动作或策略，本文的贡献对象是带有角色、权限、工具和通信位置的 agent 节点。第三，LLM-MAS 的输出是自然语言、代码、工具调用和文件修改的混合 trace，贡献受到 prompt、上下文、权限和拓扑的共同影响。因此，本文采用合作博弈式 coalition evaluation 和 removal/replacement protocol，而不是直接学习可微分的价值分解模型。

6.6 本文与已有工作的关系

表14总结了本文与主要相关方向的区别。总体而言，现有 LLM-MAS 架构工作证明了多角色协作的可行性，benchmark 工作提供了协作任务和过程指标，归因工作开始量化单个 agent 的边际贡献，MARL credit assignment 提供了长期的概念基础。本文的独特定位在于：以 agent-level contribution 为核心因变量，系统检验 topology、role、permission、trace behavior 和 task difficulty 对贡献的解释力，并通过受控反事实干预验证这些因素是否具有稳健影响。

7 总结

本文提出一个面向 LLM-MAS 的智能体贡献归因框架，用于回答“什么使一个智能体重要”。我们将多智能体系统形式化为包含图结构、角色、权限、模型配置和任务分布的对象，并将单个 agent 的贡献定义为合作博弈中的边际效用。本文不仅计算 Shapley、LOO、Banzhaf、Myerson 和 Owen 等贡献指标，还进一步分析贡献与 topology、role、permission、communication trace 和 task difficulty 的关系。为此，我们构建 MAS Architecture Zoo，在代码生成、角色分离和多场景协作数据集上进行实验，并通过混合效应回归、反事实干预和贡献预测模型验证假设。

当前版本保留了完整论文结构和结果表字段；尚未完成实验的结果单元格保持空白。完成实验后，本文将能够系统回答：degree 是否真的预测 agent importance，角色和权限是否中介拓扑效应，verifier/critic 何时产生正贡献或负贡献，LOO 何时可以近似 Shapley，以及低成本结构特征能否预测关键 agent。若这些假设得到验证，本文将为下一步 contribution-aware heterogeneous model allocation 提供实证基础。

表 14: 相关工作与本文的定位比较。

研究方向	代表工作	主要关注	与本文的关系
LLM-MAS 架构	CAMEL、AutoGen、AgentVerse、MetaGPT、ChatDev、MapCoder	构造多角色、多工具或多阶段协作系统，提高复杂任务表现。	本文将这些系统抽象为 Architecture Zoo，用于分析节点贡献来源。
多智能体 benchmark	MultiAgentBench、TeamBench、CoLLAB、CalBench、MindAgent、SOTOPIA、AgentCollabBench	评估协作、竞争、角色隔离、隐私、通信和过程失败。	本文复用其任务和过程指标，但进一步进行 agent-level attribution。
合作博弈归因	Shapley、Banzhaf、Myerson、Owen、SHAP、Data Shapley	为复杂系统中的组成单元分配边际贡献。	本文将这些方法用于 LLM-MAS 节点，并比较不同 attribution protocol。
LLM agent 贡献分析	CapaBench、Agent That Matters、Agents that Matter、HiveMind	计算模块或 agent 的贡献，识别冗余或低贡献组件。	本文从“贡献计算”进一步推进到“贡献机制解释”。
MARL credit assignment	Difference rewards、VDN、QMIX、COMA	在全局回报下学习个体动作或策略贡献。	本文借鉴反事实思想，但面向黑箱 LLM workflow 的离线诊断。

参考文献

[1] Lu, MingYu, Huang, Yushan, and Lee, Su-In. An Attribution Framework for Multi-Agent LLMs. *ICLR 2026 AIWILD Workshop*, 2026. OpenReview. <https://openreview.net/forum?id=esabw9linR>.

[2] Lu, Mingyu, Huang, Yushan, Lin, Chris, and Lee, Su-In. Agents that Matter: Optimizing Multi-Agent LLMs via Removal-Based Attribution. arXiv:2605.27621, 2026. <https://arxiv.org/abs/2605.27621>.

[3] Kim, Yubin, Park, Chanwoo, Kim, Taehan, Park, Eugene, Schmidgall, Samuel, Rahman, Salman, Park, Chunjong, Breazeal, Cynthia, Liu, Xin, Palangi, Hamid, Park, Hae Won, and McDuff, Daniel. TeamBench: Evaluating Agent Coordination under Enforced Role Separation. arXiv:2605.07073, 2026. <https://arxiv.org/abs/2605.07073>.

[4] Zhu, Kunlun, Du, Hongyi, Hong, Zhaochen, Yang, Xiaocheng, Guo, Shuyi, Wang, Zhe, Wang, Zhenhailong, Qian, Cheng, Tang, Xiangru, Ji, Heng, and You, Jiaxuan. MultiAgent-Bench: Evaluating the Collaboration and Competition of LLM Agents. arXiv:2503.01935, 2025. <https://arxiv.org/abs/2503.01935>.

- [5] Hong, Sirui, Zhuge, Mingchen, Chen, Jiaqi, Zheng, Xiawu, Cheng, Yuheng, Zhang, Ceyao, Wang, Jinlin, Wang, Zili, Yau, Steven Ka Shing, Lin, Zijuan, Zhou, Liyang, Ran, Chenyu, Xiao, Lingfeng, Wu, Chenglin, and Schmidhuber, Jürgen. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. arXiv:2308.00352, 2023. <https://arxiv.org/abs/2308.00352>.
- [6] Qian, Chen, Liu, Wei, Liu, Hongzhang, Chen, Nuo, Dang, Yufan, Li, Jiahao, Yang, Cheng, Chen, Weize, Su, Yusheng, Cong, Xin, Xu, Juyuan, Li, Dahai, Liu, Zhiyuan, and Sun, Maosong. ChatDev: Communicative Agents for Software Development. arXiv:2307.07924, 2023. <https://arxiv.org/abs/2307.07924>.
- [7] Islam, Md. Ashraful, Ali, Mohammed Eunus, and Parvez, Md Rizwan. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. arXiv:2405.11403, 2024. <https://arxiv.org/abs/2405.11403>.
- [8] Wu, Qingyun, Bansal, Gagan, Zhang, Jieyu, Wu, Yiran, Li, Beibin, Zhu, Erkang, Jiang, Li, Zhang, Xiaoyun, Zhang, Shaokun, Liu, Jiale, Awadallah, Ahmed Hassan, White, Ryen W., Burger, Doug, and Wang, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv:2308.08155, 2023. <https://arxiv.org/abs/2308.08155>.
- [9] Li, Guohao, Hammoud, Hasan Abed Al Kader, Itani, Hani, Khizbullin, Dmitrii, and Ghanem, Bernard. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. arXiv:2303.17760, 2023. <https://arxiv.org/abs/2303.17760>.
- [10] Chen, Mark, Tworek, Jerry, Jun, Heewoo, Yuan, Qiming, Pinto, Henrique Pondé de Oliveira, Kaplan, Jared, Edwards, Harri, Burda, Yuri, Joseph, Nicholas, Brockman, Greg, Ray, Alex, Puri, Raul, Krueger, Gretchen, Petrov, Michael, Khlaaf, Heidy, Sastry, Girish, Mishkin, Pamela, Chan, Brooke, Gray, Scott, Ryder, Nick, Pavlov, Mikhail, Power, Alethea, Kaiser, Lukasz, Bavarian, Mohammad, Winter, Clemens, Tillet, Philippe, Such, Felipe Petroski, Cummings, Dave, Plappert, Matthias, Chantzis, Fotios, Barnes, Elizabeth, Herbert-Voss, Ariel, Guss, William Hebggen, Nichol, Alex, Paino, Alex, Tezak, Nikolas, Tang, Jie, Babuschkin, Igor, Balaji, Suchir, Jain, Shantanu, Saunders, William, Hesse, Christopher, Carr, Andrew N., Leike, Jan, Achiam, Josh, Misra, Vedant, Morikawa, Evan, Radford, Alec, Knight, Matthew, Brundage, Miles, Murati, Mira, Mayer, Katie, Welinder, Peter, McGrew, Bob, Amodei, Dario, McCandlish, Sam, Sutskever, Ilya, and Zaremba, Wojciech. Evaluating Large Language Models Trained on Code. arXiv:2107.03374, 2021. <https://arxiv.org/abs/2107.03374>.
- [11] Austin, Jacob, Odena, Augustus, Nye, Maxwell, Bosma, Maarten, Michalewski, Henryk, Dohan, David, Jiang, Ellen, Cai, Carrie, Terry, Michael, Le, Quoc, and Sutton, Charles. Program Synthesis with Large Language Models. arXiv:2108.07732, 2021. <https://arxiv.org/abs/2108.07732>.

- [12] Jimenez, Carlos E., Yang, John, Wettig, Alexander, Yao, Shunyu, Pei, Kexin, Press, Ofir, and Narasimhan, Karthik. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. <https://arxiv.org/abs/2310.06770>.
- [13] Shapley, Lloyd S. A Value for n-Person Games. In Kuhn, H. W. and Tucker, A. W., editors, *Contributions to the Theory of Games II*, pages 307–317. Princeton University Press, Princeton, NJ, 1953.
- [14] Banzhaf, John F. Weighted Voting Doesn’t Work: A Mathematical Analysis. *Rutgers Law Review*, 19:317–343, 1965.
- [15] Myerson, Roger B. Graphs and Cooperation in Games. *Mathematics of Operations Research*, 2(3):225–229, 1977.
- [16] Owen, Guillermo. Values of Games with A Priori Unions. In Henn, Rudolf and Moeschlin, Otto, editors, *Mathematical Economics and Game Theory: Essays in Honor of Oskar Morgenstern*, pages 76–88. Springer, Berlin, 1977.
- [17] Wang, Lei, Ma, Chen, Feng, Xueyang, Zhang, Zeyu, Yang, Hao, Zhang, Jingsen, Chen, Zhiyuan, Tang, Jiakai, Chen, Xu, Lin, Yankai, Zhao, Wayne Xin, Wei, Zhe, and Wen, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. arXiv:2308.11432, 2024. <https://arxiv.org/abs/2308.11432>.
- [18] Guo, Taicheng, Chen, Xiuying, Wang, Yaqi, Chang, Ruidi, Pei, Shichao, Chawla, Nitesh V., Wiest, Olaf, and Zhang, Xiangliang. Large Language Model Based Multi-Agents: A Survey of Progress and Challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024. <https://www.ijcai.org/proceedings/2024/890>.
- [19] Chen, Shuaihang, Liu, Yuanxing, Han, Wei, Zhang, Weinan, and Liu, Ting. A Survey on LLM-Based Multi-Agent System: Recent Advances and New Frontiers in Application. arXiv:2412.17481, 2024. <https://arxiv.org/abs/2412.17481>.
- [20] Chen, Weize, Su, Yusheng, Zuo, Jingwei, Yang, Cheng, Yuan, Chenfei, Chan, Chi-Min, Yu, Heyang, Lu, Yaxi, Hung, Yi-Hsin, Qian, Chen, Qin, Yujia, Cong, Xin, Xie, Ruobing, Liu, Zhiyuan, Sun, Maosong, and Zhou, Jie. AgentVerse: Facilitating Multi-Agent Collaboration and Exploring Emergent Behaviors. arXiv:2308.10848, 2023. <https://arxiv.org/abs/2308.10848>.
- [21] Liu, Xiao, Yu, Hao, Zhang, Hanchen, Xu, Yifan, Lei, Xuanyu, Lai, Hanyu, Gu, Yu, Ding, Hangliang, Men, Kaiwen, Yang, Kejuan, Zhang, Shudan, Deng, Xiang, Zeng, Aohan, Du, Zhengxiao, Zhang, Chenhui, Shen, Sheng, Zhang, Tianjun, Su, Yu, Sun, Huan, Huang, Minlie, Dong, Yuxiao, and Tang, Jie. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=zAdUB0aCTQ>.

- [22] Ma, Chang, Zhang, Junlei, Zhu, Zhihao, Yang, Cheng, Yang, Yujiu, Jin, Yaohui, Lan, Zhenzhong, Kong, Lingpeng, and He, Junxian. AgentBoard: An Analytical Evaluation Board of Multi-Turn LLM Agents. arXiv:2401.13178, 2024. <https://arxiv.org/abs/2401.13178>.
- [23] Wang, Xingyao, Wang, Zihan, Liu, Jiateng, Chen, Yangyi, Yuan, Lifan, Peng, Hao, and Ji, Heng. MINT: Evaluating LLMs in Multi-Turn Interaction with Tools and Language Feedback. In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=jp3gWrMuIZ>.
- [24] Agashe, Saaket, Fan, Yue, Reyna, Anthony, and Wang, Xin Eric. LLM-Coordination: Evaluating and Analyzing Multi-Agent Coordination Abilities in Large Language Models. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 8088–8116, 2025. <https://aclanthology.org/2025.findings-naacl.448/>.
- [25] Gong, Ran, Huang, Qiuyuan, Ma, Xiaojian, Vo, Hoi, Durante, Zane, Noda, Yusuke, Zheng, Zilong, Zhu, Song-Chun, Terzopoulos, Demetri, Fei-Fei, Li, and Gao, Jianfeng. MindAgent: Emergent Gaming Interaction. In *Findings of the Association for Computational Linguistics: NAACL 2024*, 2024. <https://aclanthology.org/2024.findings-naacl.200/>.
- [26] Zhou, Xuhui, Zhu, Hao, Mathur, Leena, Zhang, Ruohong, Yu, Haoifei, Qi, Zhengyang, Morency, Louis-Philippe, Bisk, Yonatan, Fried, Daniel, Neubig, Graham, and Sap, Maarten. SOTOPIA: Interactive Evaluation for Social Intelligence in Language Agents. In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=mM7VurbA4r>.
- [27] Mahmud, Saaduddin, Bagdasarian, Eugene, and Zilberstein, Shlomo. CoLLAB: A Framework for Designing Scalable Benchmarks for Coordinating LLM Agents. *SEA @ NeurIPS 2025*, 2025. <https://openreview.net/forum?id=372FjQy1cF>.
- [28] Zou, Chelsea, Yao, Yiheng, She, Selena, Goodman, Noah D., and Hawkins, Robert D. CalBench: Evaluating Coordination–Privacy Trade-offs in Multi-Agent LLMs. arXiv:2605.09823, 2026. <https://arxiv.org/abs/2605.09823>.
- [29] Mazumder, Aritra, Dipta, Shubhashis Roy, Lia, Nusrat Jahan, Khan, Tanzila, Hossain, Kainat Raisa, Shri, Nehaa, Debsarkar, Shubhrangshu, Tasnim, Humayra, Shawon, Gour Gupal Talukder, Mitra, Debjoty, Rani, Sumaiya Ahmed, Anik, Al Jami Islam, and Khan, Al Nafeu. AgentCollabBench: Diagnosing When Good Agents Make Bad Collaborators. arXiv:2605.08647, 2026. <https://arxiv.org/abs/2605.08647>.
- [30] Lundberg, Scott M. and Lee, Su-In. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*, volume 30, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html.

- [31] Ghorbani, Amirata and Zou, James. Data Shapley: Equitable Valuation of Data for Machine Learning. In *Proceedings of the 36th International Conference on Machine Learning*, 2019. <https://proceedings.mlr.press/v97/ghorbani19c.html>.
- [32] Yang, Yingxuan, Huang, Bo, Qi, Siyuan, Feng, Chao, Hu, Haoyi, Zhu, Yuxuan, Hu, Jinbo, Zhao, Haoran, He, Ziyi, Liu, Xiao, Wang, Zongyu, Qiu, Lin, Cao, Xuezhi, Cai, Xunliang, Yu, Yong, and Zhang, Weinan. Who’s the MVP? A Game-Theoretic Evaluation Benchmark for Modular Attribution in LLM Agents. arXiv:2502.00510, 2025. <https://arxiv.org/abs/2502.00510>.
- [33] Xia, Yihan, Wang, Taotao, Zhang, Shengli, Weng, Zhangyuhua, Cao, Bin, and Liew, Soung Chang. HiveMind: Contribution-Guided Online Prompt Optimization of LLM Multi-Agent Systems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 34411–34419, 2026. <https://ojs.aaai.org/index.php/AAAI/article/view/40222>.
- [34] Freeman, Linton C. A Set of Measures of Centrality Based on Betweenness. *Sociometry*, 40(1):35–41, 1977.
- [35] Bonacich, Phillip. Power and Centrality: A Family of Measures. *American Journal of Sociology*, 92(5):1170–1182, 1987.
- [36] Page, Lawrence, Brin, Sergey, Motwani, Rajeev, and Winograd, Terry. The PageRank Citation Ranking: Bringing Order to the Web. Technical Report 1999-66, Stanford InfoLab, 1999.
- [37] Newman, Mark E. J. *Networks: An Introduction*. Oxford University Press, 2010.
- [38] Yan, Bingyu, Zhang, Xiaoming, Zhang, Litian, Zhang, Lian, Zhou, Ziyi, Miao, Dezhuang, and Li, Chaozhuo. Beyond Self-Talk: A Communication-Centric Survey of LLM-Based Multi-Agent Systems. arXiv:2502.14321, 2025. <https://arxiv.org/abs/2502.14321>.
- [39] Wolpert, David H. and Tumer, Kagan. Optimal Payoff Functions for Members of Collectives. *Advances in Complex Systems*, 4(2–3):265–279, 2001.
- [40] Sunehag, Peter, Lever, Guy, Gruslys, Audrunas, Czarnecki, Wojciech M., Zambaldi, Viničius, Jaderberg, Max, Lanctot, Marc, Sonnerat, Nicolas, Leibo, Joel Z., Tuyls, Karl, and Graepel, Thore. Value-Decomposition Networks For Cooperative Multi-Agent Learning. arXiv:1706.05296, 2017. <https://arxiv.org/abs/1706.05296>.
- [41] Rashid, Tabish, Samvelyan, Mikayel, De Witt, Christian Schroeder, Farquhar, Gregory, Foerster, Jakob, and Whiteson, Shimon. QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning. In *Proceedings of the 35th International Conference on Machine Learning*, 2018. <https://proceedings.mlr.press/v80/rashid18a.html>.

- [42] Foerster, Jakob N., Farquhar, Gregory, Afouras, Triantafyllos, Nardelli, Nantas, and Whiteson, Shimon. Counterfactual Multi-Agent Policy Gradients. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018. <https://ojs.aaai.org/index.php/AAAI/article/view/11794>.